Universidade Estadual de Mato Grosso do Sul

Discente: Mathews Henrique Costa da Costa

Docente: Prof. Dr. Rubens Barbosa

O objetivo deste trabalho é criar uma aplicação cliente-servidor utilizando o protocolo UDP, criando um sistema capaz de enviar o preço de vários postos de combustíveis e requisitar o preço mais barato em uma região. O sistema é composto de dois programas: servidor e cliente, os clientes podem cadastrar informações sobre postos de gasolina como tipo de combustível, preço e localização e pesquisar o posto de combustível com o menor preço para um tipo de combustível específico, dentro de um raio de busca a partir de uma coordenada.

Algoritmos e funções

pacote.h: Este arquivo de cabeçalho é o módulo central que define a estrutura de comunicação, garantindo que cliente e servidor "falem a mesma língua".

- TipoPacote (enum): Define os tipos de pacotes possíveis na comunicação:
- DADO: Para enviar informações de um posto ou o resultado de uma pesquisa.
- PESQUISA: Para requisitar uma busca por preços.
- ACK: Confirmação de recebimento sem erros (Acknowledgement).
- NAK: Notificação de recebimento com erro (Negative Acknowledgement).
- Pacote (struct): A unidade de dados trocada entre cliente e servidor. Contém o tipo do pacote, um buffer de dados (char dados[1024]) e um campo erro para simular falhas na transmissão.

dados_postos.csv: Através de uma macro, funciona como a base de dados para todos os registros de postos de combustível. O arquivo utiliza o formato CSV, onde cada linha corresponde a um único registro e seus campos são delimitados por vírgulas. A estrutura de cada linha é definida pela formatação utilizada nas funções fprintf e sscanf no código do servidor.
Cada registro é composto por cinco campos, na seguinte ordem:

- ID da Mensagem: Um identificador numérico para o registro, associado à variável msg_id.

- Tipo de Combustível: Um código inteiro (tipoCombustivel) que representa o tipo de combustível (e.g., 0 para Diesel, 1 para Álcool, 2 para Gasolina).

- Preço: O valor do combustível (preco), que é enviado pelo cliente e armazenado no arquivo como um número inteiro (preço original multiplicado por 1000).

- Latitude: A coordenada de latitude do posto, armazenada como um número de ponto flutuante.

- Longitude: A coordenada de longitude do posto, também armazenada como um número de ponto flutuante.

servidor.c: Implementa a lógica do servidor para escutar as requisições, processá-las e armazenar os dados.

<u>Principais Funções:</u>

- main(): Ponto de entrada do servidor. Sua principal responsabilidade é iniciar o socket, o endereço e o mutex. Em seguida, entra em um loop infinito que atua como um "despachante": aguarda a chegada de um pacote (recvfrom) e, para cada pacote recebido, cria uma nova thread (pthread_create) para processá-lo, delegando todo o trabalho lógico.

- request(void *args): É a função de trabalho executada por cada thread no servidor. Ela contém toda a lógica da aplicação: decodifica os argumentos da ThreadArgs, envia ACK ou NAK com base no erro simulado e processa a requisição, seja para salvar dados (D) ou para realizar uma busca (P).

- haversine(): Implementa o algoritmo de Haversine para calcular a distância em metros entre duas coordenadas geográficas (latitude e longitude).

- radianos(): Função auxiliar que converte graus para radianos.

- combustíveis(): Converte o código numérico do combustível na string respectiva do combustível.

Lógica de Operação:

Ao receber um pacote, verifica a flag de erro simulado. Se houver erro, envia um NAK.

Caso contrário, envia um ACK para sinalizar ao cliente que a requisição foi recebida corretamente.

Se a requisição for do tipo 'D' (Dados), os dados do posto são extraídos e anexados ao arquivo dados_postos.csv.

Se for do tipo 'P' (Pesquisa), o servidor lê o arquivo dados_postos.csv, e para cada posto que corresponda ao tipo de combustível, calcula a distância usando a função haversine(). O menor preço encontrado dentro do raio de busca especificado é armazenado. Ao final, um novo pacote do tipo DADO com o resultado é enviado ao cliente.

Para evitar que uma requisição de cliente (especialmente uma pesquisa, que pode ser lenta) bloqueie outras, foi adotado um modelo de uma thread por requisição. Isso permite que o servidor seja responsivo e atenda múltiplos clientes simultaneamente. A função pthread_detach é usada para que o sistema operacional lide com a limpeza dos recursos da thread quando ela termina, simplificando o código do servidor principal e o uso de pthread_mutex_t para garantir a integridade dos dados, multiplos clientes podem estar acessando ao mesmo tempo, o mutex assegura que apenas uma thread possa ler ou escrever no arquivo por vez, prevenindo condições de corrida e corrupção de dados.

cliente.c: Implementa a interface do usuário para enviar requisições ao servidor.

Principais Funções:

- main(): Configura o socket UDP. Entra em um laço onde coleta a entrada do usuário, monta o pacote e o envia, gerenciando a lógica de retransmissão.

Lógica de Operação:

O usuário escolhe entre enviar dados ('D') ou fazer uma pesquisa ('P') e insere as informações necessárias.

Um laço de repetição garante o envio confiável da requisição. A cada tentativa, o pacote é enviado e o cliente aguarda uma resposta.

Se um ACK é recebido, o laço é encerrado.

Se um NAK é recebido, a requisição é retransmitida.

Para mensagens de PESQUISA, após receber o ACK da requisição, o cliente aguarda um segundo pacote do servidor contendo o resultado da busca.

Tratamento da Retransmissão de Mensagens

O tratamento da retransmissão de mensagens é implementado através de um mecanismo de confirmação negativa (NAK), que corresponde ao protocolo RDT 2.0, lidando com pacotes perdidos.

A lógica de retransmissão no código funciona da seguinte maneira:

Laço de Controle: A retransmissão é gerenciada por um laço while (!ack_recebido). O cliente só para de retransmitir uma mensagem quando a variável ack_recebido se torna verdadeira.

Gatilho de Retransmissão (Recebimento de NAK):

Após enviar um pacote, o cliente entra em um estado de espera bloqueante.

Se o pacote recebido do servidor for do tipo NAK, o cliente imprime uma mensagem e retransmite.

Condição de Parada (Recebimento de ACK):

A retransmissão para quando o cliente recebe uma resposta do tipo ACK (confirmação).

Nesse caso, a variável ack_recebido é definida como 1, o que torna a condição do laço falsa, encerrando o ciclo de retransmissão para aquela mensagem.

Este ciclo (enviar -> esperar -> retransmitir em caso de NAK) se repete até que um ACK seja recebido, garantindo que a requisição do cliente chegue de

forma íntegra ao servidor.

Testes:

Servidor: Primeiro rodamos o servidor fornecendo a porta, a partir deste momento ele ficará aberto aguardando receber novas mensagens do cliente.

```
rgm48941@l4m18u: ~/Documentos/redes/trabalho
rgm48941@l4m18u:~/Documentos/redes/trabalho$ ./servidor 8080
Servidor escutando na porta 8080...
Os dados dos postos serão salvos/lidos de 'dados_postos.csv'
```

(Servidor aguardando mensagens)
Fonte: Próprio autor (2025)

Cliente: O cliente irá rodar o programa com o Ip do servidor e a porta em que ele está escutando, em seguida fornecer o tipo da mensagem e seus respectivos parâmetros.
Na imagem o cliente envia uma mensagem do tipo D, fornece as informações solicitadas e logo em seguida a mensagem é enviada, não ocorre erro no envio da mensagem. Então recebemos a confirmação que o pacote foi recebido.

```
rgm48941@l4m17u: ~/Documentos/redes/trabalho
rgm48941@l4m17u:~/Documentos/redes/trabalho$ ./cliente 172.26.4.147 8080

Digite o tipo de mensagem ('D' para Dados, 'P' para Pesquisa): D
Digite o ID da mensagem (inteiro): 12
Digite o tipo de combustível (0-Diesel, 1-Álcool, 2-Gasolina): 0
Digite o preço (ex: 4.449): 5784
Digite a latitude: -22.20673121612227
Digite a longitude: -54.861086631002905

Enviando requisição: "D 12 0 5784 -22.206731 -54.861087" (erro simulado: Não)...
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 2
  -> ACK da requisição recebido!

Digite o tipo de mensagem ('D' para Dados, 'P' para Pesquisa):
```

(Cliente enviou e recebeu a confirmação de recebimento. Tipo de mensagem: D)
Fonte: Próprio autor (2025)

```
rgm48941@l4m18u: ~/Documentos/redes/trabalho
rgm48941@l4m18u:~/Documentos/redes/trabalho$ ./servidor 8080
Servidor escutando na porta 8080...
Os dados dos postos serão salvos/lidos de 'dados_postos.csv'

Pacote recebido de 172.26.4.57:41530. Conteúdo: "D 12 0 5784 -22.206731 -54.8610
87"
  -> Pacote sem erro. Enviando ACK da requisição...
Conteudo:
Tipo: 12
Combustível: 0
Preço: 5784
Latitude: -22.206731
Longitude: -54.861087
     -> Dados do posto salvos em 'dados_postos.csv'
```

(Servidor recebendo a mensagem do cliente e enviando o ACK da requisição, insere as informações recebidas no arquivo)
Fonte: Próprio autor (2025)

Na imagem o cliente envia uma mensagem do tipo P, fornece as informações solicitadas e logo em seguida a mensagem é enviada, não ocorre erro no envio da mensagem. Então recebemos a confirmação que o pacote foi recebido.

```
  -> ACK da requisição recebido!

Digite o tipo de mensagem ('D' para Dados, 'P' para Pesquisa): p
Digite o ID da mensagem (inteiro): 41
Digite o tipo de combustível a pesquisar (0-Diesel, 1-Álcool, 2-Gasolina): 0
Digite o raio de busca em metros: 20000
Digite a latitude do centro da busca: -22.206771382684757
Digite a longitude do centro da busca: -54.861086631002905

Enviando requisição: "P 41 0 20000 -22.206771 -54.861087" (erro simulado: Não)..
.
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 2
  -> ACK da requisição recebido!

Aguardando resultado da pesquisa...
  -> Resposta da pesquisa recebida: "Menor preço para Diesel: R$ 5.784"

Digite o tipo de mensagem ('D' para Dados, 'P' para Pesquisa):
```

(Cliente enviou e recebeu a confirmação de recebimento. Tipo de mensagem: P)
Fonte: Próprio autor (2025)

```
rgm48941@l4m18u: ~/Documentos/redes/trabalho
rgm48941@l4m18u:~/Documentos/redes/trabalho$ ./servidor 8080
Servidor escutando na porta 8080...
Os dados dos postos serão salvos/lidos de 'dados_postos.csv'

Pacote recebido de 172.26.4.57:41530. Conteúdo: "D 12 0 5784 -22.206731 -54.8610
87"
  -> Pacote sem erro. Enviando ACK da requisição...
Conteudo:
Tipo: 12
Combustível: 0
Preço: 5784
Latitude: -22.206731
Longitude: -54.861087
     -> Dados do posto salvos em 'dados_postos.csv'

Pacote recebido de 172.26.4.57:41530. Conteúdo: "P 41 0 20000 -22.206771 -54.861
087"
  -> Pacote sem erro. Enviando ACK da requisição...
  -> Processando mensagem de PESQUISA...
     -> Enviando resultado da pesquisa para o cliente: "Menor preço para Diesel:
 R$ 5.784"
```

(Servidor recebendo a mensagem do cliente e enviando o ACK da requisição e o resultado da pesquisa)
Fonte: Próprio autor (2025)

```
Digite o tipo de mensagem ('D' para Dados, 'P' para Pesquisa): p
Digite o ID da mensagem (inteiro): 45
Digite o tipo de combustível a pesquisar (0-Diesel, 1-Álcool, 2-Gasolina): 1
Digite o raio de busca em metros: 10000
Digite a latitude do centro da busca: -20
Digite a longitude do centro da busca: -54

Enviando requisição: "P 45 1 10000 -20.000000 -54.000000" (erro simulado: Sim)...
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 3
  -> NAK recebido! Retransmitindo...

Enviando requisição: "P 45 1 10000 -20.000000 -54.000000" (erro simulado: Sim)...
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 3
  -> NAK recebido! Retransmitindo...

Enviando requisição: "P 45 1 10000 -20.000000 -54.000000" (erro simulado: Sim)...
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 3
  -> NAK recebido! Retransmitindo...

Enviando requisição: "P 45 1 10000 -20.000000 -54.000000" (erro simulado: Não)...
Aguardando ACK/NAK da requisição...
  -> Pacote recebido do servidor. Tipo: 2
  -> ACK da requisição recebido!

Aguardando resultado da pesquisa...
  -> Resposta da pesquisa recebida: "Menor preço para Álcool: R$ 4.526"
```

(Teste de envio do cliente para o servidor com erro)
Fonte: Próprio autor (2025)

Conclusão:

O projeto implementou um sistema cliente-servidor para gerenciamento de preços de combustíveis. A arquitetura demonstrou ser funcional e robusta, utilizando um protocolo de aplicação simples sobre UDP. Foi utilizada a fórmula de Haversine para cálculos geodésicos e foi feito o uso de um arquivo CSV para facilidade no armazenamento de dados, resultando em um código eficiente e de fácil compreensão. O sistema cumpre todos os requisitos propostos, permitindo o cadastro e a consulta espacial de dados de forma confiável.

Fontes:

https://community-esri-com.translate.goog/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

https://www-educative-io.translate.goog/answers/how-to-implement-udp-sockets-in-c?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc&_x_tr_hist=true

https://www.gnu.org/software/make/manual/make.html